

# Project Report: Smart Parking Management System

## 1 Author

- **Name:** Rohan Kumar
- **Roll No:** 23f2003236
- **Email:** 23f2003236@ds.study.iitm.ac.in
- **Background:** I am a Diploma-level student at IIT Madras.

To run the application, install the required Python libraries: `flask`, `flask-sqlalchemy`, `flask-migrate`, and `bcrypt`. Then, execute the `app.py` file using the command `python app.py` in the terminal.

## 2 Description

The objective was to develop a web-based parking management system using Flask as the backend framework. The application aims to streamline parking operations, similar to real-world systems like Park+ or GetMyParking, with features for both users and administrators. Users can book and release parking spots, view their parking history, and check lot availability, while admins manage parking lots, spots, and user accounts.

### 2.1 Technologies Used

1. **Flask:** Handles HTTP requests, defines routes, and renders templates for the application.
2. **Flask-SQLAlchemy:** Manages database models, queries, and transactions.
3. **Flask-Migrate:** Facilitates database schema migrations.
4. **Bcrypt:** Secures user passwords through hashing.
5. **Jinja2:** Enhances HTML templates with dynamic content rendering.
6. **SQLite:** Stores data in a lightweight, file-based database.

## 3 Database Schema Design

Each model includes an `id` field as the primary key. The schema is designed to ensure data integrity and efficient querying.

### 1. User Class

- `id`: Integer, primary key.
- `username`: String, unique, non-nullable.
- `password`: String, non-nullable (hashed using `bcrypt`).
- `email`: String, unique, non-nullable.
- `phone`: String, nullable.
- `address`: String, nullable.
- `is_admin`: Boolean, defaults to `False`.

- **Relationship:** One-to-many with `Reservation` (cascade delete).

## 2. `ParkingLot` Class

- `id`: Integer, primary key.
- `location_name`: String, non-nullable.
- `price`: Float, non-nullable (hourly rate in Rs.).
- `address`: String, non-nullable.
- `pin_code`: String, non-nullable.
- `number_of_spots`: Integer, non-nullable.
- `status`: String, non-nullable, defaults to 'Available'.
- **Relationship:** One-to-many with `ParkingSpot` (cascade delete).

## 3. `ParkingSpot` Class

- `id`: Integer, primary key.
- `spot_number`: Integer, non-nullable.
- `capacity`: Integer, non-nullable, defaults to 1.
- `occupied`: Integer, non-nullable, defaults to 0.
- `status`: String, non-nullable, 'A' (Available) or 'O' (Occupied).
- `lot_id`: Integer, foreign key referencing `ParkingLot.id`.
- **Relationship:** One-to-many with `Reservation` (cascade delete).

## 4. `Reservation` Class

- `id`: Integer, primary key.
- `vehicle_number`: String, non-nullable.
- `parking_time`: DateTime, non-nullable, defaults to current UTC time.
- `leaving_time`: DateTime, nullable.
- `parking_cost`: Float, nullable.
- `is_active`: Boolean, defaults to True.
- `spot_id`: Integer, foreign key referencing `ParkingSpot.id`.
- `user_id`: Integer, foreign key referencing `User.id`.

The relationships enable efficient access to related data (e.g., a parking spot's lot or a reservation's user) and ensure cascading deletes maintain consistency.

# 4 Architecture and Features

## 4.1 Architecture

The application follows Flask's recommended structure:

1. `app.py`: Contains all views, routes, and application logic.

2. `templates/`: Stores HTML files rendered with Jinja2.
3. `static/`: Holds CSS and JavaScript files for styling and client-side functionality.
4. `instance/`: Contains the SQLite database file (`parking.db`).
5. `venv/`: Includes Python dependencies for the virtual environment.

## 4.2 Features

1. **Initialization**: Creates a default admin user (username: `admin`, password: `admin`) if none exists.
2. **Main Page (/)**: Redirects logged-in users to their dashboard (admin or user) or displays login/register options.
3. **User Features**:
  - **Register (/register)**: Users provide username, password, email, phone, and address. Username and email must be unique.
  - **Login (/login)**: Validates credentials using `bcrypt`; redirects to user dashboard on success.
  - **Dashboard (/user/dashboard)**: Shows active and past reservations, available parking lots, and spot availability.
  - **Book Spot (/user/book)**: Users book an available spot by providing a vehicle number; prevents multiple active bookings.
  - **Release Spot (/user/release/<reservation\_id>)**: Marks a spot as available, calculates parking cost, and updates reservation.
  - **Summary (/user/summary)**: Displays total reservations, spending, average cost, and parking duration, with data for a doughnut chart showing location usage.
  - **Edit Profile (/edit\_profile)**: Users can update email and password.
4. **Admin Features**:
  - **Dashboard (/admin/dashboard)**: Shows parking lot statistics (total, occupied, available spots).
  - **Lot Management**: Add, edit, or delete parking lots and their spots (`/admin/lots/add`, `/admin/lots/edit/<lot_id>`, `/admin/lots/delete/<lot_id>`). Prevents deletion if spots are occupied.
  - **User Management**: View, add, or delete users (`/admin/users`, `/admin/add_user`, `/admin/users/delete/<user_id>`).
  - **Summary (/admin/summary)**: Provides total revenue, lot utilization, and data for a pie chart showing lot usage and earnings.
  - **Search (/admin/search)**: Searches users (by username/email/ID), lots, or spots (by location name).
  - **Spot Details (/admin/spots/<spot\_id>)**: Shows spot status and current reservation (if occupied).
5. **Cost Calculation**: Charges are based on hourly rates, rounded up to the next hour, with

a minimum 1-hour charge.

6. **Error Handling:** Uses try-except blocks for database operations and flash messages for user feedback.

## 5 Challenges and Solutions

1. **Challenge:** Managing spot count changes when editing a parking lot.
2. **Solution:** Implemented logic to handle three cases: same count (no change), increased count (add spots), or decreased count (delete spots if not occupied).
3. **Challenge:** Ensuring data consistency during booking and release.
4. **Solution:** Used cascading deletes and transaction rollbacks to prevent orphaned records or failed operations.

## 6 Video Demonstration

A video showcasing the application's functionality is available at:

<https://drive.google.com/file/d/1mIbI4iSWtHCaVxnbzQKsPBBG253F8rwT/view?usp=sharing>